

Relatório de Execução – Sistema de Gerenciamento de Livros (Árvore AVL)

Por que a AVL mantém buscas eficientes?

A eficiência da busca em uma árvore binária depende do seu balanceamento. A Árvore AVL mantém buscas eficientes porque impõe o Fator de Balanceamento, onde qualquer nó não pode ultrapassar o limite de 1 ou -1. Essa restrição assegura que a árvore mantenha sempre uma altura otimizada. Dessa forma, o tempo de busca será de $O(\log N)$ se a árvore está balanceada, tornando a consulta de livros extremamente rápida independentemente do tamanho.

Em quais situações ocorreram mais rotações?

Ao compararmos as diferentes baterias de testes realizadas, fica evidente que a proporção e a ocorrência das rotações dependem diretamente do nível de ordenação prévia dos dados inseridos.

- **Testes dos Dados Aleatórios:** No teste de carga (script principal), onde foi inserido 100 códigos gerados aleatoriamente e realizamos 20 remoções, as rotações ocorreram de forma diluída. Como a geração aleatória diminui drasticamente a chance de sequências ordenadas, a árvore exigiu ajustes pontuais e bem distribuídos entre direita (LL) e esquerda (RR) para manter o fator de balanceamento dentro do limite.
- **Testes Unitários:** A situação em que as rotações se provaram mais vitais e ocorreram na maior frequência proporcional (a cada nova inserção) foi simulada nos testes unitários de Inserção e Rotação. Ao forçar a entrada de dados em sequências desfavoráveis puras (crescentes como 10, 20, 30; ou decrescentes como 30, 20, 10), provocou-se o pior cenário de uma árvore binária: a degeneração estrutural em forma de "escada". Nessas situações, a árvore AVL disparou imediatamente as rotações simples (RR e LL) e duplas (LR e RL) para promover o nó central à raiz. Além disso, no teste de remoção, nota-se que deletar nós críticos com dois filhos (como a própria raiz) também exige um esforço maior da estrutura, pois a busca pelo nó substituto (menor da subárvore direita) desencadeia um recálculo do fator de balanceamento que sobe em cascata pela árvore.

Quais vantagens ela apresenta em relação a uma árvore binária de busca comum?

A principal vantagem da árvore AVL em relação à Árvore Binária de Busca (ABB) comum é a eliminação do pior caso de eficiência. Numa ABB comum, o algoritmo não garante que a árvore gerada a cada passo esteja balanceada, o que pode transformar a complexidade da busca em $O(N)$. A AVL resolve esse

problema de forma dinâmica através das suas rotações (Simples à Direita/Esquerda e Duplas).

Capturas da Execução do Programa (Testes Unitários)

Abaixo estão os registros dos testes isolados que comprovam a integridade matemática da estrutura desenvolvida.

```
victo@vitim da juju MINGW64 ~/Documents/Trabalho - Árvore AVL em Python
$ "C:/Program Files/Python312/python.exe" "c:/Users/victo/Documents/Trabalho - Árvore AVL em Python/src/teste_insercao_rotacao.py"

=====
TESTE: INSERÇÃO E ROTAÇÕES (LL, RR, LR, RL)
=====

1. Forçando Rotação RR (Árvore pesa para a direita)
-> Esperado: Raiz 20 | Obtido: Raiz 20

2. Forçando Rotação LL (Árvore pesa para a esquerda)
-> Esperado: Raiz 20 | Obtido: Raiz 20

3. Forçando Rotação LR (Esquerda-Direita)
-> Esperado: Raiz 20 | Obtido: Raiz 20

4. Forçando Rotação RL (Direita-Esquerda)
-> Esperado: Raiz 20 | Obtido: Raiz 20
=====
```

Figura 1: Validação das rotações LL, RR, LR e RL, garantindo que a árvore se reestrutura corretamente e promove o nó central para a raiz.

```
victo@vitim da juju MINGW64 ~/Documents/Trabalho - Árvore AVL em Python
$ "C:/Program Files/Python312/python.exe" "c:/Users/victo/Documents/Trabalho - Árvore AVL em Python/src/teste_remocao.py"

=====
TESTE: REMOÇÃO (0, 1 e 2 FILHOS)
=====

Total de nós inicial: 8

1. Removendo Nó Folha (código 5)
-> Total após remover (Esperado 7): 7

2. Removendo Nó com 1 Filho (código 10)
-> Total após remover (Esperado 6): 6

3. Removendo Nó com 2 Filhos (código 30 - RAIZ)
-> Total após remover (Esperado 5): 5

Validação: Nova Raiz após remover o 30
-> Nova Raiz (Esperado 35): 35
=====
```

Figura 2: Testes de busca exata (com sucesso e falha esperada) e validação da busca por intervalo, que percorre apenas os ramos com dados pertinentes.

```
victo@vitim da juju MINGW64 ~/Documents/Trabalho - Árvore AVL em Python
$ "C:/Program Files/Python312/python.exe" "c:/Users/victo/Documents/Trabalho - Árvore AVL em Python/src/teste_busca.py"

=====
TESTE: BUSCA EXATA E POR INTERVALO
=====

1. Busca Exata (código Existente: 30)
-> Resultado: [30] Livro 30 - Autor (2026)

2. Busca Exata (código Inexistente: 99)
-> Resultado (Esperado False): False

3. Busca por Intervalo (Entre 15 e 45)
-> Esperado: Livros 20, 30 e 40
[20] Livro 20 - Autor (2026)
[30] Livro 30 - Autor (2026)
[40] Livro 40 - Autor (2026)
=====
```

Figura 3: Comprovação da estabilidade da árvore ao remover nós-folha, nós com 1 filho e nós com 2 filhos (incluindo a substituição correta da raiz).

```
victo@victor-da-juju:~/Documents/Trabalho - Árvore AVL em Python$ "c:/Program Files/Python312/python.exe" "c:/Users/victo/Documents/Trabalho - Árvore AVL em Python/src/teste_metricas.py"

TESTE: PERCURSO IN-ORDER E MÉTRICAS
=====

1. Percurso Em Ordem (Deve imprimir: 10, 20, 30, 40, 50)
[10] Livro 10 - Autor (2026)
[20] Livro 20 - Autor (2026)
[30] Livro 30 - Autor (2026)
[40] Livro 40 - Autor (2026)
[50] Livro 50 - Autor (2026)

2. Leitura de Métricas
-> Altura da Árvore: 2 (Esperado 2)
-> Total de Elementos: 5 (Esperado 5)
-> Rotações à Esquerda (RR): 2
-> Rotações à Direita (LL): 1
=====
```

Figura 4: Leitura do percurso em-ordem (in-order) e validação dos contadores de altura, quantidade de elementos e rotações realizadas.

```
victo@vitim da juju MINGW64/Documents/Trabalho - Árvore AVL em Python
$ "C:/Program Files/Python312/python.exe" "c:/Users/victo/Documents/Trabalho - Árvore AVL em Python/src/main.py"

SISTEMA DE GERENCIAMENTO DE LIVROS - ÁRVORE AVL
=====

Inserido 100 livros aleatórios na árvore...
-> Sucesso! Total de livros na árvore: 100

Realizado 20 buscas aleatórias...
-> Concluído! Foram encontrados 20/20 livros.

Removido 20 livros da árvore...
-> Concluído! Foram removidos 20/20 livros.

[Extra] A testar a busca por intervalo (códigos entre 2000 e 2500):
[2218] Livro 2218 - Autor Desconhecido (1990)
[2272] Livro 2272 - Autor Desconhecido (2026)
[2348] Livro 2348 - Autor Desconhecido (2022)
[2408] Livro 2408 - Autor Desconhecido (2013)
[2486] Livro 2486 - Autor Desconhecido (2024)

=====

RELATÓRIO FINAL DE EXECUÇÃO
=====

Altura final da árvore:          7
Total de rotações à esquerda (RR): 31
Total de rotações à direita (LL): 32
Quantidade de elementos restantes: 80
=====
```

Figura 5: Teste-01 do projeto pronto.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
victo@victim da juju MINGW64 ~/Documents/Trabalho - Árvore AVL em Python
$ "c:/Program Files/Python312/python.exe" "c:/Users/victo/Documents/Trabalho - Árvore AVL em Python/src/main.py"

=====
SISTEMA DE GERENCIAMENTO DE LIVROS - ÁRVORE AVL
=====

Inserido 100 livros aleatórios na árvore...
-> Sucesso! Total de livros na árvore: 100

Realizado 20 buscas aleatórias...
-> Concluído! Foram encontrados 20/20 livros.

Removido 20 livros da árvore...
-> Concluído! Foram removidos 20/20 livros.

[Extra] A testar a busca por intervalo (Códigos entre 2000 e 2500):
[2032] Livro_2032 - Autor Desconhecido (1970)
[2156] Livro_2156 - Autor Desconhecido (2020)

=====
RELATÓRIO FINAL DE EXECUÇÃO
=====

Altura final da árvore:          7
Total de rotações à esquerda (RR): 34
Total de rotações à direita (LL): 30
Quantidade de elementos restantes: 80
=====

```

Figura 6: Teste-02 do projeto pronto.

```
victo@victim da juju MINGW64 ~/Documents/Trabalho - Árvore AVL em Python
$ "c:/Program Files/Python312/python.exe" "c:/Users/victo/Documents/Trabalho - Árvore AVL em Python/src/main.py"
=====
SISTEMA DE GERENCIAMENTO DE LIVROS - ÁRVORE AVL
=====

Inserido 100 livros aleatórios na árvore...
-> Sucesso! Total de livros na árvore: 100

Realizado 20 buscas aleatórias...
-> Concluído! Foram encontrados 20/20 livros.

Removido 20 livros da árvore...
-> Concluído! Foram removidos 20/20 livros.

[Extra] A testar a busca por intervalo (códigos entre 2000 e 2500):
[2120] Livro_2120 - Autor Desconhecido (1988)
[2381] Livro_2381 - Autor Desconhecido (1999)

=====
RELATÓRIO FINAL DE EXECUÇÃO
=====
Altura final da árvore:          7
Total de rotações à esquerda (RR): 31
Total de rotações à direita (LL): 31
Quantidade de elementos restantes: 80
=====
```

Figura 7: Teste-03 do projeto pronto.

```
victo@victim da juju MINGW64 ~/Documents/Trabalho - Árvore AVL em Python
$ "c:/Program Files/Python312/python.exe" "c:/Users/victo/Documents/Trabalho - Árvore AVL em Python/src/main.py"
=====
SISTEMA DE GERENCIAMENTO DE LIVROS - ÁRVORE AVL
=====

Inserido 100 livros aleatórios na árvore...
-> Sucesso! Total de livros na árvore: 100

Realizado 20 buscas aleatórias...
-> Concluído! Foram encontrados 20/20 livros.

Removido 20 livros da árvore...
-> Concluído! Foram removidos 20/20 livros.

[Extra] A testar a busca por intervalo (códigos entre 2000 e 2500):
[2267] Livro_2267 - Autor Desconhecido (1970)
[2309] Livro_2309 - Autor Desconhecido (2012)
[2462] Livro_2462 - Autor Desconhecido (2022)

=====
RELATÓRIO FINAL DE EXECUÇÃO
=====
Altura final da árvore:          6
Total de rotações à esquerda (RR): 37
Total de rotações à direita (LL): 32
Quantidade de elementos restantes: 80
=====
```

Figura 8: Teste-04 do projeto pronto.